



Analiza sprzętu i oprogramowania (open-source oraz własnego) do mapowania terenu i detekcji obiektów w bezzałogowych statkach powietrznych – raport z przeprowadzonych badań

Cel

Celem badań była analiza dostępnych bibliotek open-source, implementacja własnych rozwiązań oraz dopasowanie odpowiednich narzędzi umożliwiających mapowanie terenu oraz detekcję obiektów z wykorzystaniem bezzałogowych statków powietrznych (BSP). W ramach prac podjęto próbę opracowania własnego oprogramowania i porównania jego efektywności z istniejącymi rozwiązaniami. Dodatkowo dokonano doboru odpowiedniego sprzętu, w tym kamer oraz jednostek obliczeniowych (minikomputerów pokładowych i stacji naziemnych), do realizacji poszczególnych zadań. Misje testowe zostały zaprojektowane w oparciu o wymagania zawodów SUAS 2025 (Stany Zjednoczone) oraz TEKNOFEST 2025 (Turcja). Badania prowadzono z wykorzystaniem dwóch typów statków powietrznych – płatowca oraz wielowirnikowca – w celu oceny skuteczności rozwiązań w różnych scenariuszach operacyjnych.

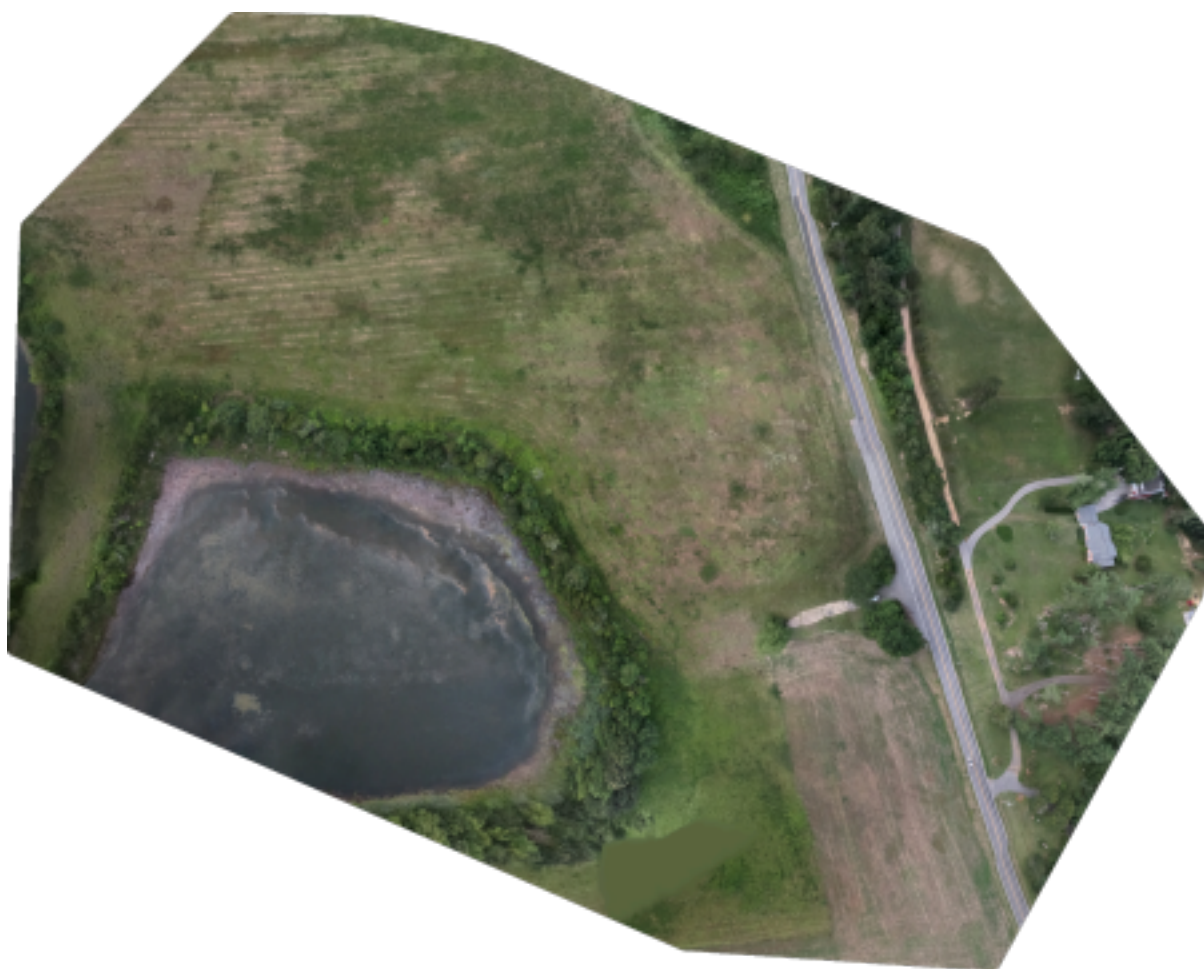
Mapowanie terenu

Przebieg badań

Jednym z zadań konkursowych podczas zawodów SUAS 2025 było zmapowanie terenu przy użyciu bezzałogowego statku powietrznego. Zespół startował z BSP w konfiguracji płatowca. Wymagało to zastosowania kamery zdolnej do nagrywania obrazu w bardzo wysokiej jakości, odpornej na zakłócenia wynikające z dużych prędkości lotu (do 30 m/s) oraz wychyleń podczas manewrów charakterystycznych dla płatowców. Z tego względu zdecydowano się na kamerę GoPro 11, która łączyła wysokie parametry obrazu z odpowiednią stabilizacją, a jednocześnie mieściła się w założonym budżecie grantu.

Pierwszym etapem prac było stworzenie prototypu autorskiego oprogramowania do przetwarzania obrazu z wykorzystaniem biblioteki OpenCV w języku Python, który sprzyja planowaniu i prototypowaniu. Zgodnie z przypuszczeniami pierwsze testy wykazały, że wydajność tego rozwiązania była niewystarczająca przy wymaganej jakości i liczbie zdjęć generowanych przez kamerę. W związku z tym właściwą implementację wykonano w języku C++, co pozwoliło na skrócenie czasu obliczeń i uzyskanie większej kontroli nad parametrami. Oprogramowanie było następnie wielokrotnie optymalizowane oraz testowane w różnych warunkach terenowych (powierzchnie trawiaste, asfaltowe, obszary o małej liczbie punktów charakterystycznych).

Przykładowa mapa wygenerowana podczas testów terenowych:



Ograniczenia sprzętowe

Kluczowym wyzwaniem okazały się wymagania sprzętowe. Ze względu na małe rozmiary kadłuba płatowca oraz jego konstrukcję dostosowaną do dużych prędkości i zwinności, instalacja wydajnego minikomputera pokładowego była praktycznie niemożliwa. Mniejsze systemy wbudowane nie zapewniały natomiast wystarczającej mocy obliczeniowej, a ich praca powodowała dodatkowe problemy – nadmierne nagrzewanie, brak miejsca na komponenty wspomagające (np. moduły GPU/AI), a także potencjalne ryzyko uszkodzeń pozostałej elektroniki. Alternatywą było wykonywanie obliczeń na stacji naziemnej podczas krótkiego, dziesięciominutowego międzylądowania. Rozwiązanie to zwiększało możliwości obliczeniowe, lecz równocześnie ograniczało czas obliczeń z 30 minut (długość całej misji) do 10 minut.

Dodatkowym problemem była ograniczona możliwość przesyłania obrazu w czasie rzeczywistym. Warunki radiowe podczas zawodów (duża liczba aktywnych sygnałów) nie gwarantowały stabilnego połączenia ze stacją naziemną, co wykluczało obliczenia „w locie”.

Dobór oprogramowania

Z uwagi na powyższe ograniczenia podjęto decyzję o wykorzystaniu narzędzia OpenDroneMap (ODM). Jest to sprawdzone, otwarteźródłowe rozwiązanie, rozwijane przez

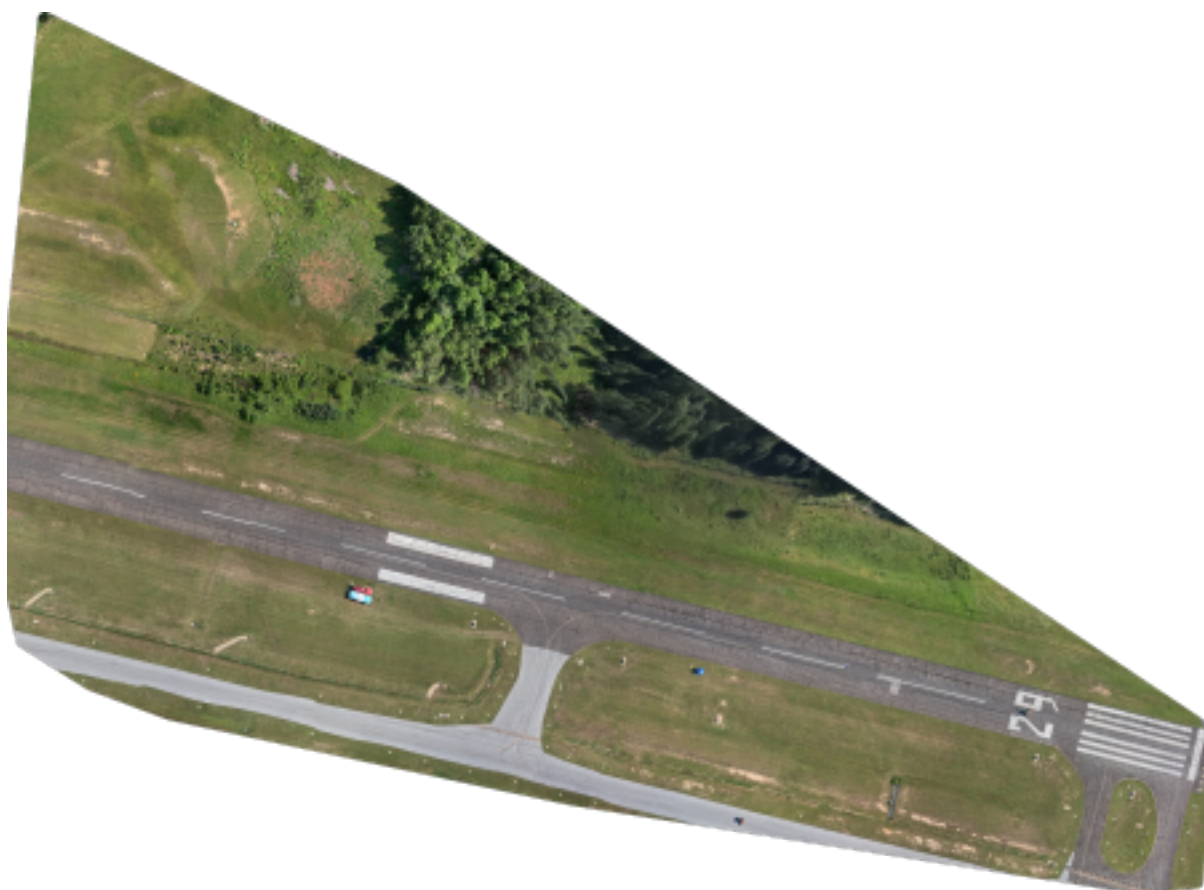


dużą społeczność specjalistów, zawierające zaawansowane i zoptymalizowane algorytmy, których implementacja w OpenCV byłaby niepraktyczna w danych warunkach. Dodatkową zaletą ODM była możliwość łatwej konfiguracji i zapisu parametrów, co pozwalało na szybkie dostosowanie procesu mapowania do charakterystyki danej misji. Obliczenia realizowano na laptopie będącym elementem stacji naziemnej podczas międzylądowań, co zapewniało kompromis między jakością map a ograniczeniami sprzętowymi.

Wyniki

Ostatecznie wygenerowane mapy charakteryzowały się bardzo dobrą jakością, a czas ich tworzenia przy wykorzystaniu ODM mieścił się w akceptowalnym limicie (poniżej 10 minut). Podczas właściwej próby konkursowej konieczna była jednak ewakuacja zespołu, co uniemożliwiło ponowne wykonanie obliczeń w czasie przeznaczonym na lot i zespół nie zdobył punktów za tę konkurencję. Sama mapa została jednak poprawnie wygenerowana już po zakończeniu misji, co potwierdziło skuteczność opracowanego rozwiązania.

Ostateczna mapa wygenerowana podczas zawodów SUAS 2025. Obszar zgodny z wyznaczonymi waypointami:



Detekcja obiektów

Przebieg badań



Podczas zawodów TEKNOFEST 2025 zespół wystawił dwie drużyny, korzystające z różnych platform BSP – pławowca oraz wielowirnikowca. Celem misji była detekcja obiektów w postaci kolorowych figur geometrycznych. W ramach prac rozważono dwa podejścia do opracowania systemu: zastosowanie sztucznej inteligencji z wykorzystaniem modelu YOLOv8, zastosowanie klasycznych metod przetwarzania obrazu przy użyciu biblioteki OpenCV.

Podejście z wykorzystaniem modelu YOLO

Na wstępnym etapie przeanalizowano dokumentację oraz przewodniki dotyczące trenowania i optymalizacji modeli YOLOv8. Do prac wykorzystano m.in. artykuły i dokumentację (przykładowo [Medium: Fine-tuning YOLO for Custom Object Detection – A Complete Guide, docs.ultralytics.com](https://docs.ultralytics.com), [Ultralytics: Hyperparameter Tuning Guide](#)). Wykorzystano także ogólnodostępne zbiory danych z platformy Roboflow Universe, które zawierały zestawy figur geometrycznych [Shapes \(ws-gybnz\)](#), [Square Circle Triangle Rectangle](#), [Geometric Shapes \(annotationnations-y2-blockb\)](#). Dane te wymagały jednak dostosowania – ograniczenia liczby klas do potrzebnych w misji oraz ręcznej edycji plików adnotacji. Uzupełnieniem zbioru były dane własne, pozyskane z nagrań wykonanych przez BSP w warunkach zbliżonych do konkursowych. Zaletami tego podejścia były: wysoka dokładność detekcji, możliwość pracy w czasie rzeczywistym oraz duża dostępność gotowych modeli i baz danych. Wadami – wysokie zapotrzebowanie na moc obliczeniową, czasochłonny proces przygotowania danych oraz konieczność żmudnego fine-tuningu modelu.

Podejście klasyczne z wykorzystaniem OpenCV

Drugą metodą było zastosowanie klasycznych technik przetwarzania obrazu, dostępnych w bibliotece OpenCV. Podejście to opierało się na segmentacji kolorów w przestrzeni HSV, wykrywaniu konturów, przybliżaniu ich do odpowiednich wielokątów i klasyfikacji kształtów na podstawie liczby wierzchołków. W momencie detekcji obraz był zapisywany i przesyłany na serwer. Zaletami tej metody były prostota implementacji, brak konieczności trenowania modelu oraz niskie wymagania sprzętowe – algorytm działał sprawnie na procesorze CPU bez użycia GPU. Wadami natomiast: konieczność ręcznej kalibracji parametrów, duża wrażliwość na oświetlenie i zmiany tła oraz podatność na fałszywe detekcje.

Wybrane rozwiązanie

Po testach przeprowadzonych na obu platformach BSP lepsze wyniki uzyskano przy wykorzystaniu podejścia klasycznego. Kluczowym czynnikiem był niski koszt obliczeniowy i możliwość uruchomienia algorytmu w czasie rzeczywistym na pokładzie BSP. Do tego celu zastosowano Raspberry Pi 5 (8 GB RAM) – najmocniejszy dostępny model, który oferował wystarczającą wydajność, przy zachowaniu kompaktowych rozmiarów i niskiej ceny. Architektura systemu obejmowała: nagranie obrazu → klatkowanie filmu → segmentacja kolorów (HSV) → detekcja konturów i wyznaczenie otoczki wypukłej → klasyfikacja kształtu na podstawie liczby wierzchołków → obliczenie środka ciężkości → przekształcenie środka na waypoint → modyfikacja trasy BSP → przesłanie zdjęcia obiektu na serwer.

Dobór kamery



Ważnym elementem systemu był wybór odpowiedniej kamery. Przetestowano m.in. GoPro 11, kamerę FPV oraz różne modele przeznaczone do współpracy z Raspberry Pi. Podczas prac badawczych testowano możliwość wykorzystania kamery GoPro 11 do systemu detekcji obiektów. Pomimo bardzo dobrej jakości obrazu oraz stabilizacji, kluczowym problemem okazała się integracja z jednostką obliczeniową Raspberry Pi. GoPro nie posiada natywnej, łatwej do implementacji metody przesyłania obrazu w czasie rzeczywistym do Raspberry Pi. Dostępne opcje, takie jak transmisja przez Wi-Fi lub HDMI, wymagały stosowania dodatkowych adapterów oraz zewnętrznych modułów. Rozwiązania te: zwiększały złożoność całego systemu i ryzyko awarii, powodowały dodatkowe opóźnienia w transmisji obrazu, co utrudniało analizę w czasie rzeczywistym, zajmowały dodatkowe miejsce w kadłubie BSP, ograniczając przestrzeń dla innych podzespołów, generowały problemy ze stabilnością połączenia – szczególnie przy jednoczesnym obciążeniu systemu innymi procesami, wpływały na większe zapotrzebowanie energetyczne i dodatkowe nagrzewanie sprzętu. Dodatkowo wysoka jakość wideo powodowała, że program musiał przetwarzać nadmiarową ilość klatek o niepotrzebnie wysokiej rozdzielczości, co spowalniało obliczenia. GoPro 11 okazała się niepraktycznym rozwiązaniem w systemie pokładowym opartym na Raspberry Pi. Ostatecznie wybrano Raspberry Pi Camera HD v3 (12 MPx). Zdecydowały o tym: jej niewielki rozmiar i łatwa integracja z Raspberry Pi, niższy koszt w porównaniu z GoPro, wystarczająca jakość obrazu do detekcji. Rozważano użycie kamery szerokokątnej, która pozwalałaby objąć większy teren, jednak ze względu na silne zniekształcenia obrazu zrezygnowano z niej na rzecz wariantu o kącie 76°. Wymagało to wydłużenia misji o dodatkowy przelot, aby pokryć teren nieuwzględniony podczas pierwszego przelotu.

Wyniki

System klasyczny oparty na bibliotece OpenCV, jednostce obliczeniowej Raspberry Pi 5 oraz kamerze Pi HD v3 okazał się stabilny i wystarczająco szybki, aby działać w czasie rzeczywistym na obu platformach BSP. Testy terenowe potwierdziły poprawną detekcję obiektów, a integracja z autopilotem umożliwiała automatyczne modyfikowanie trasy lotu na podstawie wykrytych figur geometrycznych. Podczas zawodów pojawiły się jednak nieprzewidziane trudności. Wielowirnikowiec nie ukończył wcześniejszej misji i uległ awarii (rozbiciu), co uniemożliwiło pełną weryfikację systemu detekcji na tej platformie. W przypadku płatownicy odnotowano natomiast fałszywą detekcję – system błędnie zaklasyfikował kolorowy wzór na koszulce jednego z zawodników jako obiekt docelowy. Nie wykrył też docelowych obiektów ze względu na ich powierzchnię, która intensywnie odbijała światło słoneczne przez co zmieniała się barwa obiektu. Incydent ten potwierdził wrażliwość klasycznych metod detekcji na zakłócenia wynikające ze zmiennego tła oraz obecności niepożądanych obiektów w polu widzenia kamery.